

P0020r2 : Floating Point Atomic

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0020r2
Date: 2016-03-14
Reply-to: hcedwar@sandia.gov
Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Hans Boehm
Contact: hboehm@google.com
Author: Olivier Giroux
Contact: hboehm@google.com
Author: JF Bastien
Contact: jfb@google.com
Author: James Reus
Contact: reus1@llnl.gov
Audience: SG1 Concurrency, Library Evolution
URL: <https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0020.rst>

1 Motivation

This paper proposes an extension to the atomic operations library [atomics] for atomic addition on an object conforming to the *atomic-view-concept* (see P0019, Atomic View) instantiated for a floating point type. A class conforming to the *atomic-view-concept* shall also provide the following operations when T is a floating-point type. This capability is critical for high performance computing (HPC) applications.

2 Proposal

[Note: In 3.9.1p8 floating point types are also referred to as *floating* types. - end note]

Add to 29.5, Atomic Types, before 29.5p1.

```
template<> struct atomic< floating > {
    floating fetch_add( floating operand , memory_order order = memory_order_seq_cst ) volatile noexcept;
    floating fetch_add( floating operand , memory_order order = memory_order_seq_cst ) noexcept;
    floating fetch_sub( floating operand , memory_order order = memory_order_seq_cst ) volatile noexcept;
    floating fetch_sub( floating operand , memory_order order = memory_order_seq_cst ) noexcept;
    floating operator+=( floating operand ) volatile noexcept;
    floating operator+=( floating operand ) noexcept;
    floating operator-=( floating operand ) volatile noexcept;
    floating operator-=( floating operand ) noexcept;
};

template<> struct atomic-view-concept < floating > {
    floating fetch_add( floating operand , memory_order order = memory_order_seq_cst ) const noexcept;
    floating fetch_sub( floating operand , memory_order order = memory_order_seq_cst ) const noexcept;
    floating operator+=( floating operand ) const noexcept;
    floating operator-=( floating operand ) const noexcept;
};
```

Insert into the listed types in 29.5p5:

floating specializations

Insert new paragraph after 29.6.5p30.

Remark: For *floating* types atomic arithmetic operations *should* conform to `std::numeric_limits< floating >` (18.3.2) traits associated with the floating point type. The floating point environment (26.3) for these operations may be

different than the calling thread's floating point environment.